Assignment 2 Guidelines

You will continue the OverTheWire bandit challenge up to level 11. You will also complete 4 challenges that will involve discovering a secret password.

Do this assignment on your own. The assignment is due by **March 12th, 2026 at 11:59 PM EST**. Zip each of your files into a directory with the naming convention **Lastname_Firstname.zip**. You can also use **tar.gz** (e.g., Lastname_Firstname.tar.gz).

- Make sure that your zip file can be uncompressed and the submitted code is readable with a text editor. Corrupted files will count zero points.
- A complete assignment zip file should contain 2 folders (one for each part) with the naming convention **Part_x** where x is the part's number (e.g., Part_1 not Part 1, and not Part 1 Bandit). The folders should contain each part's output files: one for part 1, and three files for part 2, following the instructions below.
- DO NOT leave environment or IDE directories in your submission. These include but are not limited to: .venv, .conda, .idea, .vscode.
- The rubric is listed at the end of this document

Part 1: Bandit_2 (20 pts)

As with the previous assignment, the goal of this assignment is to familiarize yourself with accessing a Linux environment via SSH (that we will discuss later in class), along with developing skills on command line interaction and learning about "capture-the-flag" style wargames.

To do this, you will solve the next 5 levels (in other words solve levels 6 through 11) on the overthewire.org Bandit challenges.

To track your progress, download the template of the part 1 report file, and fill in the requested sections. **Take screenshots of the series of commands you ran to find the password for each level**. Also, **take screenshots of you entering the password to reach the next level**. Your final report should include descriptions of all steps taken for logging into all accounts up to and including bandit11 on the OverTheWire bandit server.

1. A screenshot of the steps taken to obtain the password for the next level up to and including level 11 (i.e., log into level 11, but you do not need to solve for the level 12 password).
2. A screenshot showing the act of logging into all accounts up to and including bandit11 on the OverTheWire bandit server.
3. A brief description of what you did (demonstrate that you understand what the commands you ran did rather than simply restating the list of commands).

For example, the level 0 -> 1 section of your report may look like the following (without the redactions on the solutions):

1. A screenshot of steps to obtain the password for the next level (yours should not be redacted and should include the actual password and all steps to obtain it) :

2. A screenshot of successfully logging into the next level (it is ok if the password is not visible here, but there should be at least a partial banner to demonstrate that the attempt was successful. Also, include the ssh command in the screenshot.):

3. In your own words, give a description of what you did to obtain the password (do not simply list the commands, demonstrate that you understood what those commands accomplished).

Note that Bandit is an open system, and the goal of this assignment is to practice and develop your own skills, so be honorable and do not read walkthroughs. **Failure to report screenshots or manipulated screenshots will result in points deduction.**

Submission Instructions for Part 1

As before, please submit a report file with a detailed description and screenshots of the solutions for each level. This report should be included in the **Lastname_Firstname** zip file, **Part_1** folder. You can submit the report in pdf, docx, and .odt format. Points will be deducted for missing content.

Part 2: Hash cracking (80 pts)

The goal of this part of the assignment is for you to learn basic hash cracking. For this part, you will be cracking 4 different types of hashes and writing a report that contains your name, and a description of how you broke/reversed the hash (part 2 template). You will also need to include the password that you broke for each given hash (MD5Crypt, DESCrypt, bcrypt, and custom). For the custom hash, you also will submit the code and makefile. You can create this in Python or C.

Get your hashes here. The first letter of your surname and the first 2, 3, or 4 letters of your first name determine the 4 hashes for you to break (e.g., John Smith = SJO).

You are free to use any resource or program to help you solve these challenges, except for each other. For example, open-source software such as John The Ripper, or any password cracking software.

1. MD5Crypt (15 pts)

You will be cracking an MD5Crypt hash (MD5 hash with added salt so that the same plaintext may not hash the same twice).

As a check `rampazzi` hashes to

`$1$SSrUdB4k$kJKF8FfLsMtkS8kk3rDd90`

and `UFL` hashes to

`$1$r4AQ4jZA$Sz6lmOn5kt/byoJg/cRNZ0`

Our intelligence tells us this password is associated with a user who is known to recite the Bee Movie script to coworkers. Furthermore, the bank website where the password is used allows only letters and numbers.

Tips: To speed this initial test up, you may want to create a wordlist containing the words "rampazzi" and "UFL" on different lines with no spaces. If you are using John The Ripper to crack passwords, please use the md5crypt format for this part.

2. DESCrypt (15 pts)

This is a standard DESCrypt hash. This can be time-consuming, so you may want to use any available information from our intelligence to limit the search space.

As a check `rampazzi` hashes to `2Jwik/4ql52bQ`

and `UFL` hashes to `2JDPzzc8hkhf6`

Our intelligence shows that the user usually sets a 4 character password, composed of lowercase letters (a-z), uppercase letters (A-Z), and digits (0-9).

Tips: If you are using John The Ripper for this, you should go into the john.conf configuration file (possibly in /etc/john) and change the maximum and minimum search length.

3. bcrypt (15 pts)

This is a standard bcrypt hash (note that bcrypt hashes include a salt, so there are many possible outputs).

As a check `rampazzi` can hash to

`$2a$10$MPt5V7X3RDyzrE3ZJzsfCeu3n5g/WdWR4E4y.EAME7AZyy64h8oIq`

and `UFL` can hash to

`$2a$10$xOAau/ufNFAz7AdjwPsfD.WGbZ1SmOXOYXlCc02kJUV0eb6BAm.ce`

Our intelligence shows that this user may have reused a password present on the RockYou website. This database includes a compilation of compromised passwords from previous leaks.

4. Custom (35 points)

To break the custom hash, you should create a program (with Makefile) that takes the input to hash and runs it through MD5 100 times (feeding the output of MD5, as a raw digest, into the next). Then, you run that result 100 times through SHA256. Then, you run that result 100 times through SHA512. You can create this in Python or C.

In mathematical notation, something like:

```
SHA512( SHA512( ... ( SHA256( SHA256( ... ( MD5( MD5( ... ( "string"
)...)
```

Since you are writing your own code, you should also include a description of how your code works in the final report (including instructions on how to run the makefile).

As a check `rampazzi` hashes to
`c0cae648acfdec9218e1705a5f5ada4618a1cb2d68991e4725687a74e5fd5a38a155c95a50f2bf475774be0154c3def737d8e17cffc5e3132ea837f655cc2533`

`UFL` hashes to
`23a1a813edce95854cbcafc76b395cd452f057fd25d62dbec061f68541f9e346d539ee1e9c4720ebdb5a3b46fca2e30cbd2788da93d3d29a27e35585f03760d1`

`security` hashes to
`28eb8ec120b9f9fe2be8e015ad952035483b496ef476ee3004952674bb40903a0c40d55d537b10b08be9c6ea3cf08c0d8c80857cd7ae89cb67cd67e685426947`

Our intelligence tells us that the user chose a 6 character password containing letters and numbers. Our intelligence also tells us this password was also present in the RockYou database.

Hint: When encoding and decoding inputs, be careful with how you are representing data. The provided inputs should be UTF-8 encoded before being processed, and there should be no other encoding conversion in the intermediate steps. The final output should be a hexadecimal string.

Submission Instructions for Part 2

The makefile and program for the custom hash (section 4), and the report file should be included in the **Lastname_Firstname** zip file, **Part_2** folder.

You will prepare a report that includes the password that you broke and a description of how you broke that password for each hash following the given template for part 2.

Notes on the makefile:

- Do not use Pip or PipInstaller if submitting a Python makefile.
- Include the instructions on running your code on the report file (including Python version and external libraries are used).
- Your executable must be called **crack_hashes**.
- Points will be deducted **if the makefile does not create an executable file**, the **executable file contains errors**, or the generated **executable file is not used for the code execution** (see also the rubric below for details).
- The makefile should run using the command **make -B**.

If you didn't solve one of the 4 parts, leave blank the template's section related to that part. You can submit the report in pdf, docx, and .odt format. The code should be readable with a text editor.

Notes on the code:

- **DO NOT** leave **environment or IDE directories** in your submission. These include but are not limited to: **.venv, .conda, .idea, .vscode, etc.** Your submission should only include the directories we specified (Part_1 and Part_2).
- Your code must accept the hashes as command line arguments. Your code will be tested by running **./crack_hashes <custom_hash>.**
- Points will be deducted if the code does not accept input as arguments, if the hashes are hardcoded, or if the executable is not called crack_hashes.
- **Do not zip every part individually.** Only one directory needs to be zipped, and that's the **LastName_FirstName** directory with your entire submission.

Rubric:

Part 1: Bandit_2 (20 pts)

- Log into level 6 (2 pts)
- Find the password for level 7 (2 pts)
- Log into level 7 (2 pts)
- Find the password for level 8 (2 pts)
- Log into level 8 (2 pts)
- Find the password for level 9 (2 pts)
- Log into level 9 (2 pts)
- Find the password for level 10 (2 pts)
- Log into level 10 (2 pts)
- Find the password for level 11 (2 pts)

Points will be deducted for missing or forged content.

Part 2: Hash cracking (80 pts)

1. **MD5Crypt (15 pts)**

   Correct and complete description of the commands and method used to crack the hash (7.5 pts)

   Correct password (7.5 pts)

2. **DESCrypt (15 pts)**

   A correct and complete description of the commands and method used to crack the hash (7.5 pts)

   Correct password (7.5 pts)

3. **bcrypt (15 pts)**

   A correct and complete description of the commands and method used to crack the hash (7.5 pts)

   Correct password (7.5 pts)

4. **Custom Hash (35 pts)**

   Correct password (7 pts)

   Correct Makefile (7 pts)

   Correct code (7 pts)

   A correct and complete description of the method used and what the code does (including instructions to run the code) (7 pts)

   Code matches the described method in the report (7 pts)

Points will be deducted if 1) The code does not compile, 2) The Makefile is incorrect, or it doesn't recompile the code if something changes in the code 3) Part of the code is missing or incorrect 4) Part of the code is copied or contains unrelated code.